© 2026 IEEE. This is the author's accepted version of an article published in the Proceedings of the 30th IEEE International Symposium on Distributed Simulation and Real Time Applications (DS-RT 2026). The final version is available at: https://doi.org/10.1109/DS-RT71368.2026.11726810

Kubernetes-Based Real-Time LoRaWAN Emulation Toward Network Digital Twins

Alessandro Aimi, Michele Brigadoi, Andrea Sabbioni, Armir Bujari
Department of Computer Science and Engineering
University of Bologna, 40136 Bologna, Italy
Email: name.surname@unibo.it; andrea.sabbioni5@unibo.it

***Abstract*—LoRaWAN has become a widely adopted Low Power Wide Area Network technology for massive IoT deployments, but evaluating large-scale scenarios remains challenging because real testbeds with thousands of devices are costly, difficult to reproduce, and limited in controllability. This work presents a Kubernetes-based framework for supporting large-scale real-time emulation of LoRa networks using NS-3 and ELoRa. The proposed approach combines detailed LoRaWAN radio and MAC-layer modeling with transparent interaction between simulated Gateways and real LoRaWAN Network Server (LNS) components. By relying on NS-3 TapBridge integration, each emulated Gateway preserves its own network identity when forwarding UDP-encapsulated LoRaWAN traffic to external services, enabling the LNS to operate without awareness of the underlying simulation environment. Kubernetes is used to automate deployment, replication, resource isolation, and orchestration of multiple emulation instances, making the framework suitable for scalable experimentation, network digital-twin construction, and infrastructure stress testing. To assess the approach, we evaluate representative LoRaWAN emulation use cases, including network partitioning, multi-antenna configurations, and parallel configuration variants, while measuring the resource consumption, synchronization drift, server performance under high synthetic traffic, and multi-cell partitioning feasibility. Overall, the framework enables reproducible, scalable, and integrated emulation campaigns, supporting studies on adaptive data rate, server scalability, and large-scale LoRaWAN deployment.**

***Index Terms*—Network Emulation, LoRa, Digital Twin, Network Digital Twin.**

I. Introduction

Long Range Wide Area Networks (LoRaWANs) have established themselves as one of the reference technologies for large-scale Internet of Things (IoT) deployments. Their ability to provide long-range connectivity with limited energy consumption makes them suitable for a wide set of applications, including environmental monitoring, smart agriculture, smart cities, industrial sensing, logistics, and remote infrastructure supervision. In these scenarios, end devices are often battery-powered, geographically distributed, and expected to operate for long periods with minimal maintenance. The combination of long communication range, low device complexity, and operation in unlicensed spectrum has therefore made LoRaWAN an attractive solution whenever dense sensing coverage must be achieved without relying on costly or energy-demanding communication infrastructures [1].

However, the same characteristics that make LoRaWAN appealing also make its large-scale behavior difficult to evaluate. A LoRaWAN deployment is not only determined by the number of devices and gateways, but also by their spatial distribution, radio propagation conditions, channel usage, spreading factor allocation, duty-cycle limitations, interference, packet collisions, and the behavior of the LoRaWAN Network Server (LNS). These elements interact in non-trivial ways. For instance, increasing the number of end devices does not simply increase the amount of traffic handled by the infrastructure; it may also change collision patterns, affect gateway reception diversity, alter the effectiveness of LNS strategies, and expose bottlenecks in the network-server stack [2]. As a consequence, results obtained from small deployments or laboratory testbeds cannot be directly extrapolated to city-wide or industrial-scale networks composed of thousands of devices.

This challenge grows when LoRaWAN infrastructures evolve over time. Real deployments are rarely static: devices are added, gateways repositioned, traffic patterns shifted, and server parameters retuned, so operators must assess the impact of such changes before applying them to the live system. The concept of a network digital twin is valuable here [3]: an executable representation of the infrastructure able to reproduce relevant dynamics, interact with real management components, and support controlled experimentation on alternative configurations. For LoRaWAN, such a twin must go beyond coverage maps or synthetic traffic generators, which capture neither the interaction between radio access, gateway reception, and server-side control [4] nor the wireless conditions that determine which packets are received, lost, or duplicated. It must instead preserve the end-to-end chain: devices transmit over an emulated radio channel, gateways forward traffic, the LNS processes the frames, and its control decisions feed back into subsequent behavior.

Real-time network emulation offers a promising foundation for such a twin: unlike purely analytical or offline approaches, it lets an emulated network interact with external software components while respecting wall-clock timing. At large scale, however, a single instance may become insufficient when the scenario contains many devices, several gateways, or high traffic intensity. As simulated transmissions grow, so does the computational effort to process radio events, interference, reception, and forwarding, and once it becomes too high the emulator can no longer keep pace with real time, undermining its interaction with external components. Digital-twin workflows also demand repeated experiments and controlled

parameter variation, for which manual deployment of instances is inadequate.

We address these challenges with a framework for large-scale real-time emulation of LoRa networks on Kubernetes, aimed at making LoRaWAN emulation more scalable, reproducible, and suitable for digital-twin experimentation. Rather than running each emulation as a manual standalone process, the approach places scenarios inside a managed environment where instances can be deployed, parametrized, replicated, monitored, and combined as the experiment requires. Kubernetes serves as the orchestration layer, simplifying management of these workloads and supporting parallel execution of multiple replicas.

The proposed framework is designed with the interaction between the emulated access network and the real LoRaWAN infrastructure in mind. In particular, the emulation must remain compatible with an external LNS, so that the server can be evaluated under traffic conditions produced by realistic LoRaWAN access-network dynamics. The framework further lets a deployment be represented at different granularities: joint emulation where inter-gateway interference matters, or independent partitions where server-side load or spatial separation dominates, exposing the trade-off between fidelity, scalability, and cost that is central to building network digital twins. We experimentally evaluate the behavior of the framework by considering resource consumption, real-time execution limits, the impact of replicated emulation workloads on the LNS, and the feasibility of partitioning multi-cell LoRaWAN scenarios. Through these contributions, the paper provides a step toward practical tools for constructing and operating LoRaWAN network digital twins.

The remainder of the paper is organized as follows. Sections II and III provide the necessary background on LoRaWAN and NS-3 real-time emulation mechanisms and review related work on LoRaWAN simulation, real-time emulation, and cloud-based simulation frameworks. Sections IV and V present the framework architecture and implementation, along with the use cases it enables, including network partitioning, multi-antenna scenarios, and parallel configuration evaluation. Section VI reports the experimental evaluation, and Section VII concludes the paper and outlines future research directions.

II. Background

A. LoRa and LoRaWAN

Long Range (LoRa) is a proprietary radio modulation technology developed by Semtech for long-range low-power communications. In recent years, it has attracted the attention of both researchers and industrial actors, establishing itself as a valid solution for long-range outdoor sensing in both urban and remote locations. LoRa is based on Chirp Spread Spectrum (CSS) frequency modulation, where the symbol spread can be tuned to increase the signal's resistance to noise at the expense of data rate and energy consumption. At higher Spreading Factors (SFs), transmissions can be received over multiple kilometers, albeit with limited reliability [2].

Since the introduction of LoRa, the LoRa Alliance has worked to establish an open standard for LoRaWANs, defining a combined Link and Medium Access Control (MAC) network layer on top of the physical one represented by LoRa. Technical literature classifies LoRaWAN as a prominent technology among Low Power Wide Area Networks (LPWANs): the usage of free and unlicensed Industrial, Scientific and Medical (ISM) bands, combined with a comparatively simple protocol stack, has contributed to its low deployment cost and energy consumption and, ultimately, to its current widespread adoption [4].

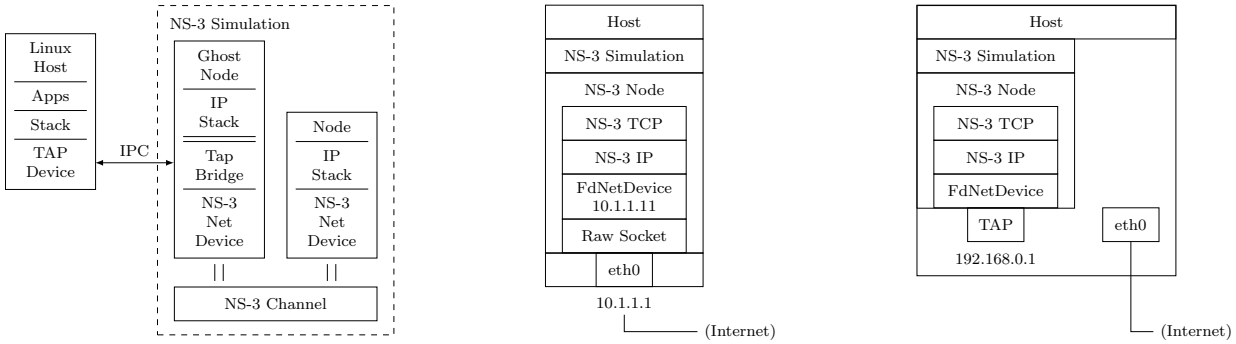
LoRaWAN adopts a centralized topology often called *star-of-stars*: End Devices (EDs) broadcast uplink frames over the radio medium, which are detected and demodulated by gateway access points; Gateways then act as transparent forwarders of LoRaWAN traffic towards a central LoRaWAN Network Server (LNS) over an IP network. This means that full LoRaWAN frames received over the radio interface are directly encapsulated in UDP or another high layer protocol to be sent to the server and vice versa. The LNS processes the LoRaWAN MAC header, checking packet integrity and dispatching its encrypted contents (the application payload) to the correct external application server.

EDs may be programmed to use one of three operation modes - Class A, B or C - to allow for downlink frame reception. Classes are ordered by increasing energy consumption while providing more communication opportunities. In Class A, they are required to open two short reception windows after sending each uplink frame; Class B adds periodical reception windows on top of the latter behavior; finally, in Class C, EDs continuously listen to the radio channel, which is the most energy-costly mode.

Downlink frames may contain application data for the ED sent by an application server through the LNS. Independently, the LNS can also send MAC commands to optimize the device usage of the radio medium. According to the LoRaWAN specifications, the LNS may control the ED's uplink SF, Transmission Power (TP), available transmission channels, number of redundant re-transmissions, maximum duty-cycle and reception windows parameters. The set of server-side techniques used to control these parameters is referred to as Adaptive Data Rate (ADR), whose optimization remains an open research problem, especially in large networks.

B. Real-Time Simulation with NS-3

The NS-3 discrete-event network simulator provides a set of emulation primitives that allow simulated nodes to exchange traffic with real-world network entities, bridging the gap between purely synthetic simulation and full physical testbed deployment. Two distinct abstractions are offered for this purpose: the `TapBridge` model and the `FdNetDevice` model, shown in Fig. 1. Although both mechanisms expose the simulated network to the host operating system, they operate at different layers of the networking stack and are suited to fundamentally different integration scenarios.



(a) TapBridge (b) FdNetDevice + Raw Socket (c) FdNetDevice + TAP Device

Fig. 1. NS-3 network emulation tools, adapted from [5]: (a) `TapBridge` connects a real host to a simulated NS-3 channel via a Linux TAP device and a ghost node; `FdNetDevice` lets simulated nodes (b) inject packets through raw sockets with MAC spoofing, or (c) appear as a host TAP device requiring bridging or NAT.

1) TapBridge: The `TapBridge` model operates at the data link layer and is designed to transparently extend an NS-3 CSMA segment into the host Linux environment. It creates a TAP virtual network interface in the host kernel and bridges it to a designated `CsmaNetDevice`, so that the TAP interface appears to the operating system as a regular Ethernet NIC. Frames arriving at any node on the simulated CSMA segment are forwarded across the bridge and injected into the host network stack; conversely, frames sent by the host through the TAP interface are delivered into the simulation as if transmitted by the bridged node.

A key architectural property of `TapBridge` is that it preserves Layer 2 addressing end-to-end. Simulated nodes retain their own MAC and IP addresses, which are the ones visible in frames egressing through the TAP interface. This transparency is essential when the communicating peer must observe traffic as originating from specific, stable source addresses without any awareness that those addresses belong to simulated entities. The host kernel's routing facilities are then available to deliver that traffic to any reachable destination, whether a local process or a remote service.

`TapBridge` supports three operating modes; the most relevant here is *ConfigureLocal*, in which NS-3 creates and configures the TAP interface autonomously, assigning it the MAC and IP address of the bridged NS-3 node and thereby eliminating manual host-side network configuration. The *UseLocal* and *UseBridge* modes instead reuse a pre-created TAP device or Linux bridge.

2) FdNetDevice: The `FdNetDevice` model instead replaces the NS-3 device abstraction with an arbitrary user-supplied file descriptor which can be a raw socket, TUN/TAP device, or promiscuous physical interface. Lacking the L2 bridging of `TapBridge`, it provides no automatic MAC-level forwarding across a multi-node segment: traffic from other simulated nodes must be routed explicitly and all addressing configured manually. It therefore suits single-exit hardware-in-the-loop scenarios rather than exposing an entire simulated LAN, making it less appropriate for our multi-gateway setup.

III. Related Work

In this section, we first review the state of the art in LoRa/LoRaWAN simulation and traffic generation to motivate the framework design. Then, we review and compare Cloud-based distributed simulation frameworks from the literature, highlighting the gaps addressed in this work. Finally, we discuss the real-time LoRaWAN emulation tool *ELoRa*, presenting an overview of its features.

A. LoRa/WAN Simulation Models and Implementations

Driven by growing interest in LoRa and LoRaWANs, researchers have investigated network behavior at both the physical and MAC layers. The hardware required to build small-scale testbeds, comprising few gateways and EDs, is generally readily available and affordable; this has allowed researchers to test, model and validate the networks' small-scale dynamics. However, small-scale testbeds are generally not able to saturate LoRa radio and hardware resources in order to evaluate large-scale dynamics and highlight potential limitations. Furthermore, the difficulties and costs involved in building and deploying large-scale real-life testbeds comprising thousands of devices have led to the development of various LoRa and LoRaWAN simulators.

Simulation models from the literature present different degrees of abstraction and feature-completeness [6]. Among them, the majority provides a radio interference model designed around a limited set of evaluation metrics, and a bespoke implementation of selected LoRaWAN protocol mechanisms. More detailed models have been developed as modules for well-known network simulation environments, such as NS-3 and Omnet++, which provide advanced tools for topology generation, radio channel modeling and packet serialization. Notable simulation models in this category are FLoRa for Omnet++ (currently maintained, no development of new features) [7] and the `lorawan` module of NS-3 (currently maintained, in active development) [8].

These tools aim to accurately model the radio performance of devices and gateways, while also providing detailed im-

plementations of the MAC protocol, but do not cover in detail the core network functions, in particular LoRaWAN network servers and bridging functions. On the other hand, only high-level traffic generators currently exist for network servers [9] without, however, the possibility of producing non-trivial network management scenarios involving complex radio channel dynamics (path-loss, capture effect on collision, interference from SF quasi-orthogonality).

B. Cloud-based Distributed Simulation Frameworks

Automating and distributing network simulations across cloud infrastructures represents an important step toward the realization of network digital twins. Several efforts in the literature have addressed this goal.

In [10], containerization of NS-3 simulations with Docker results in improved memory management in case of leaks thanks to resource isolation, also contributing to overall experiment reproducibility. Simulations are run in parallel over multiple nodes using `etcd` for coordination and `fleetctl` to launch and manage tasks as systemd units. Simulations are not run in real-time and network traffic does not exit the simulation (no interaction with real-world nodes), as the framework is designed to automate and parallelize NS-3 simulation campaigns. Similarly, the authors in [11] propose Network Simulator as a Service, a high-level architecture using Docker to run simulation scenarios in parallel. This is still not real-time emulation connected to networks outside the simulation sandbox.

The authors of [12] propose a system integrating Kubernetes focused on large-scale satellite simulation. However, real-time execution is not considered in the scope of the work. The work in [13] proposes a Kubernetes architecture for deploying and managing simulated scenarios among ad-hoc VMs and pods exchanging synthetic traffic. This work targets softwarized network functions, and no hardware model or sophisticated discrete-event simulator is involved, thereby limiting scenarios to those executable on the available hardware. With the framework as presented, it would not be possible to test networks involving a simulated wireless channel or additional physical nodes.

In [14], researchers focus on 5G O-RAN to build and compare testbeds at different levels of integration between physical deployment, real-time emulation, and simulation. They observe that real-time emulation provides greater flexibility at the cost of sometimes working with out-of-date implementations of standards in complex technological ecosystems such as 5G. However, they do not touch on the topic of testbed automation or distributed deployment.

In this work, we propose a Kubernetes deployment model for LoRaWAN real-time emulation. On the one hand, Kubernetes is instrumental in enabling automation in both deployment distribution and parallel execution of custom emulations. On the other, we base our contribution on ELoRa [15], a tool for real-time LoRaWAN emulation able to seamlessly connect simulated access networks with core software LoRaWAN functions. We also expand on it by motivating a containerization model for ELoRa and enriching its observability tools up to cloud-native standards.

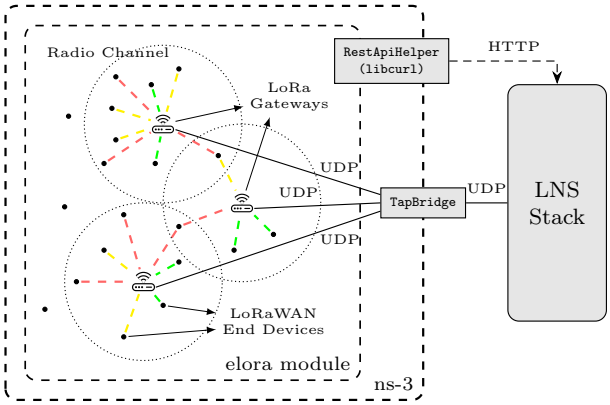


Fig. 2. Key elements of the ELoRa architecture for real-time compatibility and communication with LNS stacks. In ELoRa, simulated LoRa gateways forward UDP-encapsulated LoRaWAN traffic received from the radio channel to a `TapBridge` NS-3 node. This bridge element acts as a new network interface of the operating system currently running ELoRa, allowing UDP traffic to flow outside the real-time NS-3 simulation and vice-versa. Traffic can then be processed by a LNS instance, allowing transparent management of the simulated LoRaWAN access network. An additional ELoRa component, the `RestApiHelper`, is used to automate the registration process of gateways and devices in the LNS at the start of the emulation.

C. Real-time LoRaWAN Emulation with ELoRa

ELoRa is a real-time simulation module for LoRa/WAN built as an extension of the `lorawan` NS-3 module. It integrates real-time execution capabilities on the host machine, as well as a number of improvements to the internal protocols' model [15], needed for transparent communication between simulated devices and real LoRaWAN network functions outside the emulated environment. Fig. 2 provides an overview of ELoRa's key software components.

This approach enables real-time communication between devices and LNSs, thus allowing scaled testing of realistic LoRa traffic on core LoRaWAN functions, as well as zero-cost prototyping of radio management algorithms on a simulated radio channel. For example, the LNS is able to actively steer radio communication parameters of simulated devices through the LoRaWAN MAC protocol, affecting access and contention over the simulated LoRa channel. Overall, ELoRa provides a practical environment for investigating LoRa/LoRaWAN constraints and forms a suitable foundation for developing network digital twins.

IV. Framework Architecture and Implementation

In this work, we expand on the capabilities of ELoRa by further developing the emulator for containerized Kubernetes deployment, declarative replication and observability. In the following, we first discuss design considerations related to ELoRa's containerization, and then we present the enhancements to the emulator developed for the integration in Kubernetes.

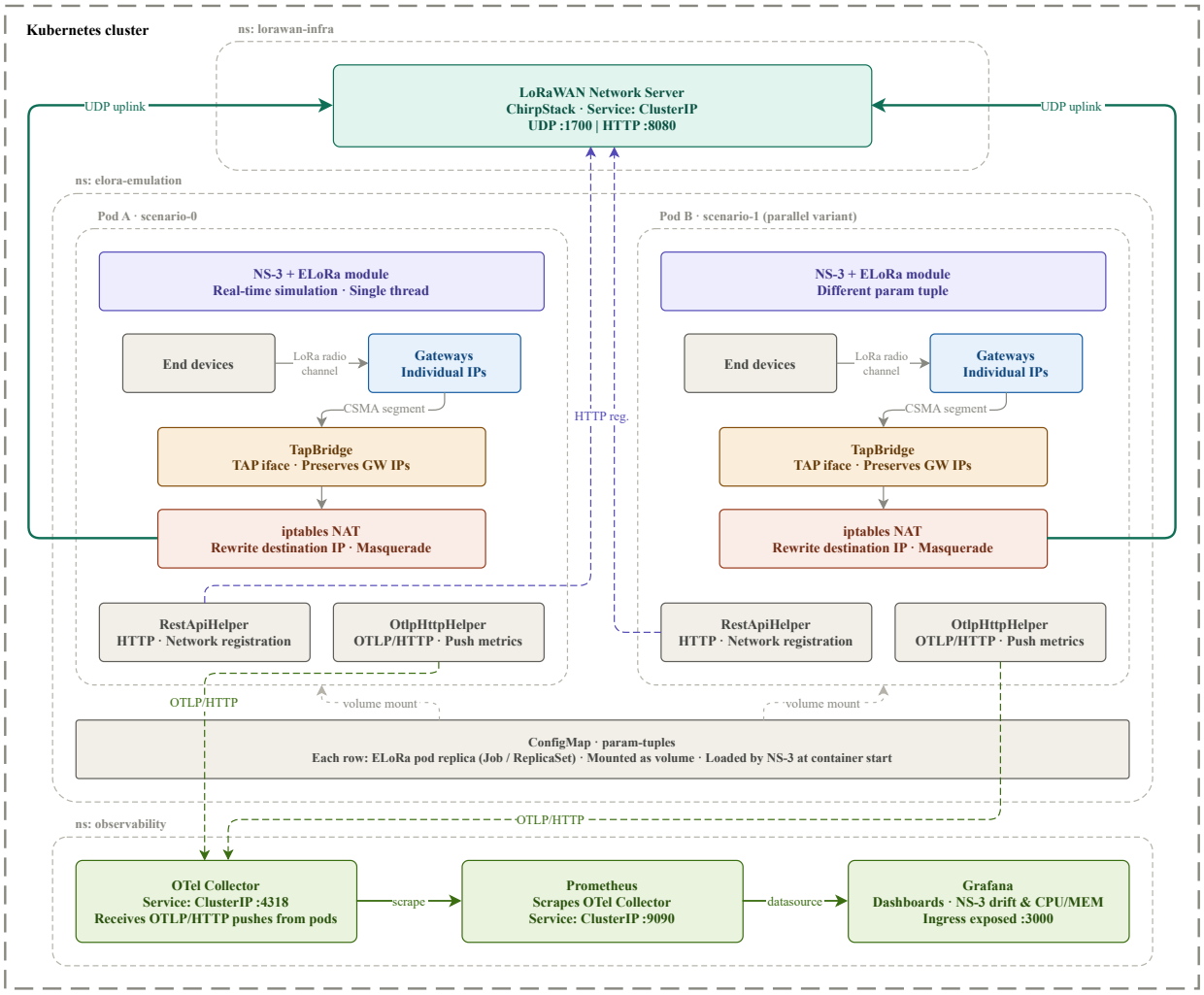


Fig. 3. Kubernetes-based deployment architecture. Each NS-3 execution is single-threaded due to the shared radio channel model among devices in the simulation, while customized parallel scenarios can be declaratively deployed thanks to Kubernetes replication feature.

A. *Containerization Design Considerations*

The emulation framework presented in this work connects multiple simulated LoRaWAN gateways to a real network server over UDP. Each gateway is modelled as a distinct NS-3 node bearing its own IP address, and the network server must receive datagrams that appear to originate from those individual gateway addresses in order to correctly associate uplink frames with the corresponding network infrastructure elements.

The NS-3 `TapBridge` model satisfies this requirement directly. By bridging the CSMA backhaul segment, to which all simulated gateways are attached, to a TAP interface on the host, the model ensures that UDP datagrams forwarded by any gateway egress with that gateway's IP address intact, without any address translation. When deployed on the same host, the network server is therefore fully agnostic to the simulated nature of its peers and requires no additional configuration, which is an explicit design goal of ELoRa.

However, communication with a network server on a different node introduces additional complexity: configuring the UDP traffic destination to a different address than the `TapBridge` requires manually setting routing tables for all NS-3 gateways for packet forwarding through the exit node; similarly, directly attaching gateways to host interfaces with the `FdNetDevice` model either requires to set the underlying physical interface in promiscuous mode for operation with raw sockets, or manual Network Address Translation (NAT) for each gateway TAP created by NS-3.

The simplest solution is to preserve the original ELoRa `TapBridge` design with a simulated CSMA segment and to configure NAT for the single exit node instance. The container ships a full precompiled installation of NS-3 with the ELoRa module, where custom scenario files (specified as the C++ `main` function, following NS-3 design) are provided as a volume and promptly compiled/linked at container startup. Here, before running the simulation, we use `iptables` to rewrite the destination address with one provided by the user via the configuration file, as well as to masquerade the source

IP addresses of the simulated gateways.

B. Enhancements for Kubernetes Integration

The architecture of our Kubernetes deployment model is shown in Fig. 3. In containerized form, ELoRa can be directly deployed in a Kubernetes cluster where an LNS is present. The IP address (or domain name) of the server is provided via a configuration file, as well as the desired simulation scenario. Any simulation parameter can be exposed and provided via the configuration file in order to customize the emulated network nodes and behavior. Specifically, we exploit this mechanism to enable the deployment of distinct parallel scenarios: users can write tuples of parameters in a file, which is then used to deploy a custom Kubernetes replica in parallel for each configuration tuple, all attached to the same LNS.

NS-3 provides several tracing mechanisms which can be used to export file-based metrics at runtime. However, file-based tracing does not integrate naturally with common Kubernetes observability tools, such as OpenTelemetry, Prometheus and Grafana. Thus, we extended ELoRa with an `OtlpHttpHelper` which implements the OpenTelemetry HTTP protocol using cURL C libraries. This enables writing custom metrics to a distributed endpoint at runtime, as we later show for the simulation drift in Section VI.

In the current implementation, emulations can be configured to run for a fixed duration or indefinitely. The LNS function can be configured to apply a static ADR policy to devices, or it can be extended for dynamic control via network API. As in actual deployments, commands available in the LoRaWAN MAC protocol are then sent to devices through the correct gateway selected by the LNS. MAC commands affect multiple parameters related to the device access to the radio channel, with potential effects on channel congestion and network performance.

V. Use Cases

The proposed framework is designed to support a range of experimental scenarios that reflect practical challenges in the evaluation and deployment of large-scale LoRaWAN infrastructures. The following use cases highlight how these capabilities can be leveraged to improve scalability, accelerate experimentation, and support data-driven design decisions in both research and operational contexts.

A. Network Partitioning

The framework accelerates large-scale campaigns by decomposing a LoRaWAN scenario into independent partitions whenever full radio coupling is not required. This suits deployments with natural spatial or logical separation, geographically distant regions sharing one backend, mobile nodes crossing areas, or frequency-based segmentation where devices on orthogonal channels are already weakly coupled at the radio level. Distributing partitions across Kubernetes-managed instances enables concurrent execution while preserving realistic interaction with shared components such as the LNS, reducing the execution time and bottlenecks of monolithic simulations at the cost of no longer enforcing a single collision domain. This approach significantly reduces execution time and computational bottlenecks associated with monolithic simulations, while still capturing the essential system-level behavior needed for performance evaluation and digital-twin workflows.

B. Multi-Antenna Scenario

The framework also supports multi-antenna deployments, where each instance represents a distinct physical configuration with different antenna parameters, placement, or radio settings. Through ELoRa's transparent integration with the LNS, these heterogeneous gateways interact simultaneously with the same backend, enabling end-to-end evaluation of reception conditions, gateway diversity, and server-side dynamics such as ADR and load balancing. Antenna parameters and traffic profiles can also be reconfigured at runtime, allowing controlled experimentation under varying load and deployment strategies. This capability is particularly valuable for assessing the impact of antenna deployments on packet reception, gateway diversity, and server-side processing dynamics, including ADR mechanisms and load balancing strategies. Furthermore, the framework allows dynamic reconfiguration of antenna parameters and traffic profiles during runtime, enabling controlled experimentation with varying load conditions and deployment strategies.

C. Parallel Emulation Support

The approach allows the execution of multiple configuration variants in parallel, varying antenna setups, device behaviors, traffic patterns, or protocol parameters, enabling systematic comparison without sequential experimentation. Kubernetes orchestration runs these variants concurrently and reproducibly, which is particularly useful for digital-twin *what-if* analysis: alternative configurations or scaling strategies can be evaluated simultaneously to identify trade-offs between performance, reliability, and resource consumption before changes are applied to a real deployment. Our framework can thus enable a more agile and data-driven approach to network design and optimization, supporting timely decision-making in evolving large-scale IoT infrastructures.

VI. Experimental Evaluation

We test our deployment on a 5-node Kubernetes cluster on the SLICES-RI infrastructure [16]. The cluster is equipped with OTel Collector agents, a Prometheus database and Grafana endpoint as shown in Fig. 3. Targeting reproducibility and stress testing, we select an ad-hoc simulation scenario where each instance consists of 7 gateways over an area of 128 km^2 as in [15]. The end-device traffic pattern is configured to generate a 5-byte packet every 5 s, representing a worst-case scenario for the LNS as channel occupancy approaches the limits imposed in the EU. As a LNS stack implementation for the experiments, we adopt the open-source tool ChirpStack [17]. Measurements are collected for 10 min after network registration, yielding at least 40 samples per

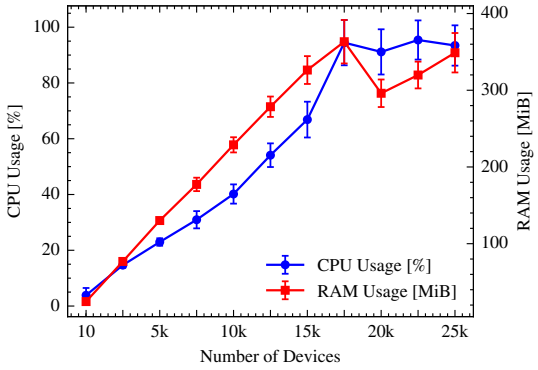


Fig. 4. CPU and memory usage of a single network emulation container against an increasing number of simulated devices.

data point at full capacity; mean values are plotted with 95% Confidence Intervals (CI).

As a first step, we evaluate CPU and RAM consumption as the number of devices in a single emulation instance increases, also monitoring the LNS stack function itself, as a high synthetic traffic load may reveal unexpected bottlenecks. This test is also one of the intended use cases of our contribution, as it can help forecast the resource allocation needs for the network server in case of expected device growth.

Then, we proceed to assess the emulation ability to keep up the synchronization of the internal simulation time model with real time. For this, we use the internal NS-3 scheduler to trace the host time passed after each simulated second, and compute the difference as the instantaneous simulation time drift. We also show how partitioning the simulated devices into multiple replicas can help mitigate the overall simulation time drift, at the cost of relaxing the assumption of a single radio collision domain.

A. Deployment Performance

We evaluate CPU and memory usage for a single emulation instance (Fig. 4) and of the LNS stack (Fig. 5). Values are collected over multiple experiments by gradually increasing the number of simulated devices, that is, the amount of traffic generated in the simulation and the overall computational load.

For the emulation in Fig. 4, resource consumption increases approximately linearly up to 17.5k devices populating the network, whereas the CPU usage stabilizes between 85% and 100%. This is expected and indicates that the simulation has exhausted its computational resources. As for most radio channel simulation models, the LoRa channel object must be accessed by all simulated physical nodes: it acts as a single point of passage, limiting multi-threaded parallelization to a single thread. This trade-off can be mitigated through modeling heuristics, albeit at the expense of some accuracy in interference-related computations.

As the simulation reaches 100% CPU utilization, the NS-3 engine struggles to keep up with the rate of events it needs to process. This has several effects: Fig. 4 shows a drop in memory usage due to lower generation of packets which, in NS-3, are memory-intensive objects. Afterwards, memory usage still grows, albeit at a lower rate, due to the larger number of statically preallocated device objects.

In Fig. 5, the LNS stack shows stable CPU activity and linear memory growth after 5k devices. When the emulation reaches 100%, both processes are altered (CPU usage drops and memory stabilizes), which highly correlates with the bottleneck in simulated events. Furthermore, it must be noted that the majority of the memory allocation for ChirpStack comes from its Postgres backend database, storing non-volatile data on the current network state, and Redis, storing logs on recent packet activity. The latter is especially affected by the packet rate in terms of memory usage, together with the main LNS function for packet processing with regard to CPU usage.

B. Emulation Time-drift and Network Partitionability

We further investigate the real-time capabilities of the NS-3 simulator by measuring its instantaneous time drift as explained in the beginning of this section. Note that the drift can be negative: we observe that, under congestion, NS-3 applies a form of preemptive event scheduling to mitigate delays with real-world interfaces.

As shown in Fig. 6, the mean simulation time drift starts growing noticeably after 10k devices due to the computational load. The growth accelerates quickly with the number of devices, reaching an average of 15s at 25k, and the overall variance increases as well. Up to 7.5k devices, we measure a CI upper bound of less than 10ms, and when the CPU reaches 100% at 17.5k we see a sudden increase of 3s compared to the previous measurement at 15k. Comparing with the result of Fig. 4, the results indicate that real-time synchronization remains accurate at moderate loads but degrades rapidly once CPU saturation is approached.

The simulation drift problem can be addressed by offloading devices to multiple emulations instances, effectively partitioning them among different sub-networks controlled by the same LNS. In Fig. 7, we distribute 25k devices among multiple emulation instances and analyze the time drift change. Device offloading to two replicas drastically cuts the positive drift; however, a certain amount of stress is still visible as the NS-3

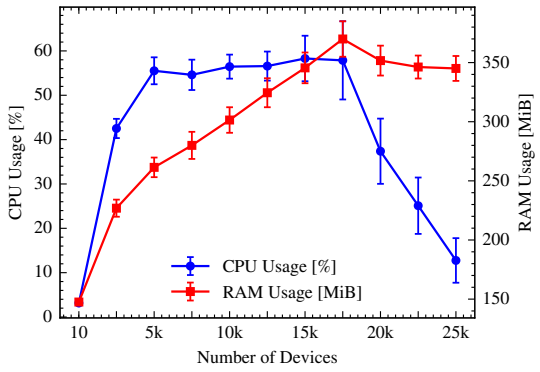


Fig. 5. CPU and memory usage of the LNS stack against an increasing number of simulated devices in a single emulated network instance.

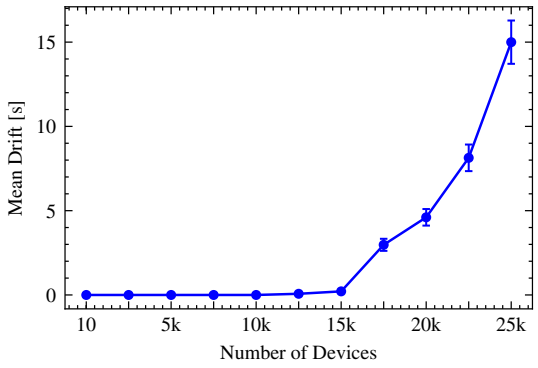


Fig. 6. Real-time drift of the ns-3 engine running in the `elora` container against an increasing number of simulated devices. To this effect, the number of end-devices contributes to a higher number of packets being sent in the simulated LoRa channel, while the number of gateways multiplicatively contributes to the number of interference events being recorded. Ultimately, these factors increase the computation time needed to process transmission events, possibly causing the simulation to lag behind and desynchronize.

engines resort to preemptive scheduling indicated by negative drift and a large CI upper bound of around 12ms. Interestingly, additional replicas further reduce the amount of preemptive scheduling applied by the NS-3 engine, landing on a maximum of 330µs at 5k devices.

This shows that network partitioning over multiple emulations can be an effective way to improve the real-time accuracy and surpass the limitations of single-threaded radio simulations by more effectively exploiting available compute power. The drawback is that devices do not share a single collision domain, limiting the interference effects in very congested, real-time scenarios. In that case, a viable alternative is to revert to a fully simulated approach [14].

VII. Conclusion

This paper presented a Kubernetes-based framework for large-scale real-time emulation of LoRaWAN networks using NS-3 and ELoRa. The framework enables containerized deployment, declarative replication, observability through cloud-native monitoring tools, and transparent interaction with real LoRaWAN Network Server components. Experimental results showed that a single emulation instance scales up to several thousand devices before real-time drift becomes significant, while partitioning the workload across multiple replicas substantially reduces synchronization drift and improves scalability. As future work, we will focus on supporting advanced runtime features, including pause-and-resume functionality, state export and restoration, emulation-state updates, and the dynamic injection of devices or configuration changes during execution. These extensions aim to introduce a feedback loop between the emulated environment and the real network infrastructure, paving the way toward adaptive LoRaWAN network digital twins capable of continuously reflecting, evaluating, and optimizing evolving deployments.

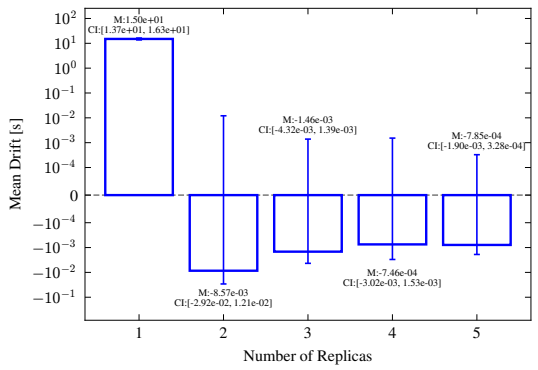


Fig. 7. Average real-time drift (symlog scale) for 25k simulated devices partitioned over an increasing number of emulation replicas.

References

[1] A. Aimi, A. Sabbioni, A. Bujari, P. Bellavista, and P. Manzoni, "CLUES: Community-driven LoRa networks urban sensing empowered by smart homes," *IEEE Netw.*, vol. 40, no. 3, pp. 264–271, May 2026.

[2] P. Gkotsiopoulos, D. Zorbas, and C. Douligeris, "Performance determinants in LoRa networks: A literature review," *IEEE Commun. Surv. & Tut.*, vol. 23, no. 3, pp. 1721–1758, Jun. 2021.

[3] P. Almasan *et al.*, "Network digital twin: Context, enabling technologies, and opportunities," *IEEE Commun. Mag.*, vol. 60, no. 11, pp. 22–27, Nov. 2022.

[4] M. Jouhari, N. Saeed, M.-S. Alouini, and E. M. Amhoud, "A survey on scalable LoRaWAN for massive IoT: Recent advances, potentials, and challenges," *IEEE Commun. Surv. & Tut.*, vol. 25, no. 3, pp. 1841–1876, May 2023.

[5] *NS-3 Network Simulator v3.47*. (2008). nsnam. Accessed: May 21, 2026. [Online]. Available: https://www.nsnam.org

[6] C. Milarokostas, D. Tsolkas, N. Passas, and L. Merakos, "A comprehensive study on LPWANs with a focus on the potential of LoRa/LoRaWAN systems," *IEEE Commun. Surv. & Tut.*, vol. 25, no. 1, pp. 825–867, Dec. 2022.

[7] M. Slabicki, G. Premsankar, and M. Di Francesco, "Adaptive configuration of LoRa networks for dense IoT deployments," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp. (NOMS)*, 2018, pp. 1–9.

[8] *LoRaWAN ns-3 module v0.3.6*. (2017). SIGNET Research Group. Accessed: May 21, 2026. [Online]. Available: https://apps.nsnam.org/app/lorawan

[9] C. Santoro and F. F. Santoro, "LWN simulator - A LoRaWAN network simulator," in *Proc. IEEE WETICE*, 2023, pp. 1–6.

[10] A. Chistyakov, "A software architecture for large multi-simulation experiments over ad hoc networks using ns-3 discrete-event network simulator," in *Proc. Eur. Model. Symp. (EMS)*, 2014, pp. 403–408.

[11] K. Antonopoulos, D. Ringas, E. Christopoulou, G. Keramidas, C. Antonopoulos, and N. Voros, "NS3SaaS: Cloud-based "Network Simulator as a Service" with customisable resource scheduling," in *Proc. IEEE CloudNet*, 2022, pp. 93–100.

[12] Y. Lai, Y. Liu, Y. Wang, K. Zhu, and J. Li, "DSatSim: A large-scale distributed satellite simulation system based on a microservices architecture," in *Proc. IEEE IC-NIDC*, 2025, pp. 353–357.

[13] M. Antzek, C. Dietz, and G. D. Rodosek, "KubeNSF: A large-scale network topology simulation framework on Kubernetes," in *Proc. Int. Conf. Netw. Future (NoF)*, 2025, pp. 186–190.

[14] J. Navarro, A. Castro, and C. Rattaro, "O-RAN testbeds: From experimental deployments to simulation frameworks," in *Proc. Wireless Days Conf. (WD)*, 2025, pp. 1–7.

[15] A. Aimi, S. Rovedakis, F. Guillemin, and S. Secci, "ELoRa: End-to-end emulation of massive IoT LoRaWAN infrastructures," in *Proc. IEEE/IFIP NOMS*, 2023, pp. 1–3.

[16] A. Sabbioni, A. Bujari, and P. Bellavista, "The Slices Cloud Continuum Blueprint: A resilient infrastructure to support large-scale Cloud Continuum experiments," in *Proc. IFIP Networking*, 2025, pp. 26–29.

[17] *ChirpStack v4.16*. (2022). ChirpStack. Accessed: May 21, 2026. [Online]. Available: https://www.chirpstack.io